

The Kinematics of Learning: Mapping and Intervening in the Geometric Stabilization of Transformer Matrices

Dror Kernan

Tel Aviv University
drorkernan@mail.tau.ac.il

Roe Lazarovich

Tel Aviv University
roeel3@mail.tau.ac.il

Code, run records and results: <https://github.com/sparrow-k/NLP-project---learning-kinematics>

Abstract

Layer freezing and layer-wise learning rates assume that different parts of a network finish learning at different times. We ask whether this also happens inside a single Transformer block, and whether a training schedule built on it can speed up training. In Phase 1 we follow every attention and feed-forward (FFN) matrix of Pythia-70m, 160m and 410m through 29 released checkpoints, and record when each gets halfway and 90% of the way to its final direction. Attention matrices settle bottom-up at all three sizes, and inside the attention block the output matrix settles first. Attention and the FFN settle together in the smaller models, while in the largest the FFN tends to settle first by one of our two measures. Weight change slows gradually rather than stopping at a clear point. In Phase 2 we train 36 small GPT-NeoX models on WikiText-103 with per-layer learning-rate schedules built from these measurements. Emphasising the FFN early reaches the target perplexity 4.4% faster than the baseline in every seed, even when each group’s total learning rate is matched, while emphasising attention early is slower. Freezing attention, by contrast, saves only 2.5% of training time and no memory, for much worse perplexity.

1 Introduction

A Transformer block has two main parts. The attention matrices W_Q, W_K, W_V, W_O decide which positions a token reads from, and the feed-forward (FFN) matrices W_{in}, W_{out} act like a key-value memory that decides what is written back (Vaswani et al., 2017; Geva et al., 2021; Meng et al., 2022). Earlier work found that networks settle from the bottom up: lower layers reach their final representations before upper ones (Raghu et al., 2017; Chen et al., 2023). Layer freezing (Brock et al., 2017; Liu et al., 2021) builds on this idea, and other methods, such as layer-wise learning rates (Howard and Ruder, 2018) and layer dropping (Zhang and He,

2020), also treat layers differently, but they all work with whole layers. It is not known in which order the parts *inside* a block settle, or whether that order can be used during training.

Research question. *During pretraining, when does each weight matrix of a Transformer block converge to its final direction, and can a per-matrix learning-rate schedule derived from that measurement improve optimization relative to an identical baseline?*

We approach this question in two phases. In Phase 1 we analyze the intra-block weight kinematics of a transformer network during training. We utilize training checkpoints of weight matrices released by the Pythia suite (Biderman et al., 2023); by flattening each matrix and using cosine similarity between the checkpoint weights to the final trained matrix, we record when it gets halfway and 90% of the way to its stabilized state. Phase 2 is an intervention. We turn these times into per-layer learning-rate schedules, train small models from scratch with them, and compare against a baseline with the same initialization and data order.

Most of our original predictions were not supported. The main findings are:

- Attention matrices settle bottom-up at all three model sizes. For the FFN, the direction of the depth trend depends on how settling is measured.
- Inside a block, attention does not settle before the FFN, and W_O — not the query/key pair — settles first.
- No matrix stops changing at a clear point. The rate of change falls smoothly, so any “stabilization step” is a choice of threshold.
- Emphasising the FFN early trains faster in every seed, also when the total learning rate is matched. Emphasising attention early is worse. Freezing matrices saves almost no compute at this scale.

All numbers in the paper are generated from the result files in our repository.

2 Related Work

When do layers settle? Raghu et al. (2017) compared network representations across training using SVCCA and found that lower layers converge to their final representations before upper ones. They suggested freezing layers once they converge (“freeze training”). Kornblith et al. (2019) proposed CKA, a more reliable similarity measure for the same kind of comparison, and Chen et al. (2023) studied the effect directly and attribute it to the flatter loss landscape of shallow layers. These works compare whole layers. We ask the same question for each weight matrix inside a layer, which can be computed from released checkpoints alone. Frankle et al. (2020) and Achille et al. (2019) showed that the first part of training has a large effect on the final network, which is why our schedules differ mainly in what happens early.

Training dynamics from checkpoints. Pythia (Biderman et al., 2023) releases 154 checkpoints per model, sampled densely early in training, to support this kind of analysis. Studies of training dynamics have followed how memorization develops over training (Tirumala et al., 2022), and found that some capabilities appear suddenly (Olsson et al., 2022), even when the mechanism behind them forms gradually (Nanda et al., 2023). These works study what the model can do; we track how its weight matrices move.

Attention and FFN do different jobs. Geva et al. (2021) showed that FFN layers act as key–value memories, and Geva et al. (2022) that they promote concepts in the vocabulary space. Meng et al. (2022) located factual associations in middle-layer FFN modules. Elhage et al. (2021) split an attention head into a QK circuit, which decides where the head attends, and an OV circuit, which decides what it writes. This split is why we compare (W_Q, W_K) against (W_V, W_O) in H3.

Schedules that use layer structure. ULMFiT (Howard and Ruder, 2018) gives each layer its own learning rate. FreezeOut (Brock et al., 2017) freezes layers one by one and removes them from the backward pass, saving up to 20% of wall-clock time on some convolutional networks. AutoFreeze (Liu et al., 2021) decides during fine-tuning which layers to freeze, and progressive layer dropping

(Zhang and He, 2020) cuts BERT pretraining cost by about a quarter. All of them act on whole layers, and the freezing and dropping methods save time by skipping computation. We test whether acting on single matrices inside a block is useful.

Change from the proposal. The proposal planned to use LMEnt (Gottesman et al., 2026). LMEnt’s main addition is metadata about the pre-training data, which a question about weight geometry does not need. What we needed most was very dense checkpoints early in training, at several model sizes: Pythia releases checkpoints at steps 1, 2, 4, ..., 512 before switching to every 1,000 steps, and our Phase 1 relies on these early steps. We therefore used Pythia; the research question did not change, and LMEnt was not used.

3 Hypotheses

We define stabilization as *directional convergence in weight space*: a matrix has stabilized when its flattened weights point in nearly the same direction as at the end of training (Section 4.2). Before processing any checkpoint we wrote down five predictions (H1-H5). Each is tested in Section 4.4, and most turned out to be wrong or only partially right.

- H1 Bottom-up.** We expected lower layers to settle before higher ones, as earlier work found for whole layers. Tested by correlating layer index with settling time.
- H2 Attention before FFN.** We expected attention (“routing”) to settle before the FFN (“memory”) in the same layer. Tested by comparing the two groups layer by layer.
- H3 W_Q, W_K before W_V, W_O .** We expected the matrices that decide where to look to settle before the ones that decide what is moved. Tested by comparing the four attention matrices.
- H4 Embeddings last.** We expected the input and output embeddings to keep changing the longest. Tested by ranking their settling times against all other matrices.
- H5 Same at every size.** We expected the orderings above to be the same in all three models. Tested by comparing results across sizes.

H2 and H3 matter most for Phase 2: treating matrices separately only makes sense if matrices inside one block settle at different times. Phase 2 hypothesis asks whether a schedule built from the measured times trains faster than an identical baseline.

4 Phase 1: Measuring Stabilization

4.1 Setup

We use pythia-70m (6 layers, $d = 512$), pythia-160m (12 layers, $d = 768$) and pythia-410m (24 layers, $d = 1024$). All three were trained on the same data in the same order for 143,000 steps. As we have limited computational resources, we use selected 29 checkpoints: [0, 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1000, 2000, 3000, 4000, 6000, 8000, 12000, 16000, 24000, 32000, 48000, 64000, 80000, 96000, 112000, 128000, 136000, 143000]. We expect volatile weight changes early in the training, so we start with a dense spacing of the checkpoints, and lower it as we go. These numbers were chosen by trial and error (first attempt used 19 checkpoints, but didn't enable enough granularity to analyze the behavior).

In every layer we track six weight matrices: W_Q , W_K and W_V , each covering all attention heads together, the attention output matrix W_O , and the FFN matrices W_{in} and W_{out} (biases are not included). We also track the input and output embeddings, which gives 38, 74 and 146 matrices for the three models.

The GPT-NeoX (Black et al., 2022) implementation used by Pythia stores Q , K and V in one fused matrix whose rows are grouped by attention head, not as three blocks. We therefore reshape it to $(n_{\text{head}}, 3, d_{\text{head}}, d)$ before splitting.

4.2 Metrics

Let W_t be a tracked matrix at step t , $W_{t'}$ the same matrix at the previous checkpoint, and W_* the matrix at step 143,000. We record

$$\cos_t = \cos(\text{vec}(W_t), \text{vec}(W_*)), \quad (1)$$

$$v_t = \|W_t - W_{t'}\|_F / \|W_t\|_F \quad (2)$$

$\cos_t$ says how close the matrix direction is to its final direction. Different matrices start at different values of $\cos_0$, so we normalize it to the *directional progress* $p_t = (\cos_t - \cos_0) / (1 - \cos_0)$, which goes from 0 at initialization to 1 at the end. The stabilization times t_{50} and t_{90} are the steps at which p_t first reaches 0.5 and 0.9, interpolated between checkpoints on a log-step axis.

Two stabilization times. We use two settling times because they answer different questions: t_{50} is when a matrix is halfway to its final direction, and t_{90} is when it has almost arrived. They do not

always agree, so we report both and call a result robust only if it holds under both. Measuring t_{90} needs checkpoints late in training. Since $p_t = 1$ at the final checkpoint by definition, a large last gap would turn t_{90} into a guess inside that gap. We therefore use checkpoints at most 16,000 steps apart up to the end, and with this grid no t_{90} value falls in the last gap. The Phase 2 schedules use t_{50} .

Rate of change. v_t is measured between consecutive checkpoints, and the gaps between them range from 1 to 16,000 steps, so raw v_t values cannot be compared. We divide by the gap and use the displacement per 1000 optimizer steps, $r_t = v_t / ((t - t') / 1000)$. We say that a matrix has “stopped” changing if its speed has dropped below a threshold τ and stayed below that threshold until the end of the training run. $t^*(\tau)$ is the first checkpoint after which r_t stays below τ ; we try τ from 0.02 to 1.6.

4.3 Statistical design

The matrices of one layer are trained together, so they are not independent samples. Counting all 144 per-layer matrices of Pythia-410m as separate samples would make the results look much more certain than they are. We therefore let each layer contribute exactly one data point to tests H1, H2 and H3. What that data point is depends on the hypothesis:

- **H1 (depth):** a single number, the median settling time of the layer's six matrices.
- **H2 (attention vs FFN):** a pair of numbers, the median time of the layer's four attention matrices and the median time of its two FFN matrices.
- **H3 (inside attention):** a set of four numbers, the settling times of the layer's W_Q , W_K , W_V and W_O .
- **H4 (embeddings):** the embeddings belong to no layer, so for each embedding we report the percentage of all other tracked matrices that settle before it.
- **H5 (across sizes):** for each model and matrix type we take the settling times along depth, resampled at 11 evenly spaced points from the bottom to the top of the model so that models with 6, 12 and 24 layers can be compared, and correlate these profiles between models.

H4 and H5 are descriptive: we report percentages and correlations without a significance test.

This leaves only $n = 6, 12$ and 24 data points for the three models, which is too small to use normal distribution fit. For H1 we check whether the single numbers rise with layer number, using a rank correlation (Spearman’s ρ , from -1 to $+1$); its p -value comes from shuffling the layer order many times and counting how often the shuffled data show as strong a trend. For H2 we test whether one member of the pair is consistently earlier across layers (Wilcoxon signed-rank test), with an effect size r from -1 to $+1$, where 0 means no difference. For H3 we first test whether the four matrices differ at all (Friedman test), then compare (W_Q, W_K) with (W_V, W_O) in the same way as H2, and finally compare every pair of matrices, adjusting the p -values because six comparisons are made (Holm correction).

4.4 Results

H1: bottom-up for attention, not for the FFN. Overall, deeper layers settle later, but clearly so only at 160m; at 70m the six layers are too few for a significant result. Figure 1 shows why the trend is weak elsewhere. The attention matrices settle later in deeper layers at every size and under both measures, while the FFN does not: in upper layers it gets halfway to its final direction early but does not finish early. Under the halfway measure this pulls the overall trend down, most visibly at 410m. The bottom-up ordering is therefore a property of attention.

H2: not supported. Attention does not settle before the FFN at any size. At 70m and 160m the two settle at the same time. At 410m the FFN is earlier by the halfway measure but not significantly by the 90% measure, so we do not treat it as a real effect.

H3: not supported; W_O settles first. The last row of Table 1 is mostly positive, the opposite of the prediction, but this comes from W_O alone. At 410m W_O reaches halfway after 8,187 steps (median over layers), against 12,877–14,869 for the other three, and it is significantly earlier than each of them under both measures. W_V is not earlier than W_Q and W_K . At 160m this holds only for the halfway measure, and at 70m not at all. So the attention output matrix settles first, but there is no general “where to look” before “what to move” ordering.

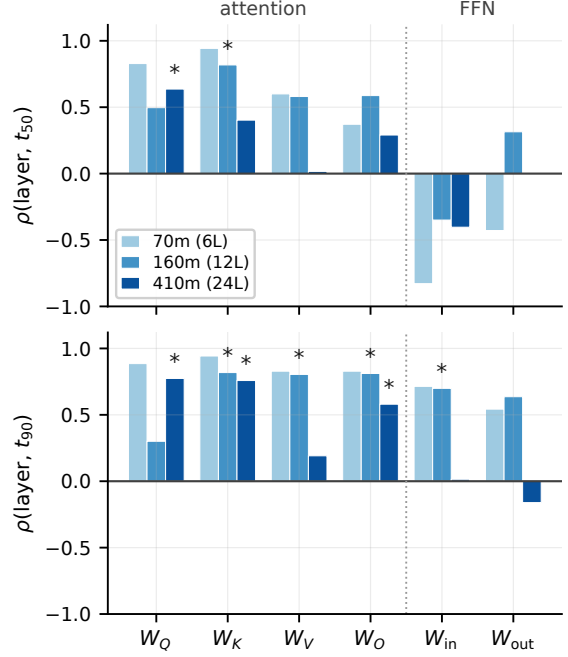


Figure 1: How strongly deeper layers settle later, for each matrix type and model size (top: halfway, bottom: 90%). Positive means deeper layers settle later. *: statistically significant after correcting for six matrix types. Attention is positive under both measures; W_{in} is negative at halfway and positive or flat at 90%.

H4: only for the output embedding, at the two smaller sizes. Table 2 shows that at 70m and 160m the output embedding is the last matrix to settle under both measures, while the input embedding is not late. At 410m the answer depends on the measure: both embeddings are among the earlier matrices at halfway but among the later ones at 90%. So the two embeddings do not behave as one group, and only the output embedding supports H4.

H5: partly supported. To compare model sizes, we correlate the depth patterns of settling times between two models: 1 would mean the same pattern, 0 no relation. The correlation is $+0.76$ between 70m and 160m, $+0.57$ between 160m and 410m, and $+0.35$ between 70m and 410m. Models of similar size therefore behave similarly, and the similarity drops as the size difference grows. In Table 1, the direction of each result is the same for all three sizes at halfway, but not always at 90%.

Weight change slows gradually. No matrix stops changing at a clear point. The rate of change falls smoothly, by about $50\text{--}80\times$ over training. Almost every matrix eventually stays below 2%

Statement tested	Pythia-70m (6 layers)		Pythia-160m (12 layers)		Pythia-410m (24 layers)	
	halfway	90%	halfway	90%	halfway	90%
(H1) Deeper layers settle later: all matrices	+0.71	+0.77	+0.83*	+0.79*	+0.22	+0.59*
(H1) Deeper layers settle later: W_Q only	+0.83	+0.89	+0.50	+0.30	+0.64*	+0.77*
(H1) Deeper layers settle later: W_{in} only	-0.83	+0.71	-0.35	+0.70*	-0.40	+0.01
(H2) The FFN settles before attention	+0.05	-0.14	+0.21	+0.46	+0.65*	+0.39
(H3) W_V, W_O settle before W_Q, W_K	+0.71	-0.71	+0.97*	+0.18	+0.85*	+0.93*

Table 1: Phase 1 results. “Halfway” and “90%” are the two settling times (t_{50} and t_{90}). Each value runs from -1 to $+1$: positive means the statement holds, negative means the opposite holds, and 0 means no pattern. *: statistically significant ($p < 0.05$; the W_Q and W_{in} rows are corrected for testing six matrix types). H2 and H3 are phrased so that a positive value is the *opposite* of what we predicted.

Model	Input embedding		Output embedding	
	halfway	90%	halfway	90%
Pythia-70m	5	84	100	100
Pythia-160m	12	52	100	100
Pythia-410m	23	83	6	89

Table 2: H4: how late each embedding settles, as the percentage of all other tracked matrices that settle before it (100 = the last of all, 0 = the first of all).

change per 1000 steps (146 of 146 at 410m), but only after step 48,000. A looser limit gives a much earlier “stopping” step, so any stabilization step depends on the limit chosen.

5 Phase 2: Intervening

5.1 Experimental setup

We train GPT-NeoX models with the pythia-70m configuration (6 layers, $d = 512$, 8 heads, rotary embeddings (Su et al., 2024); 70.4M parameters) from scratch on WikiText-103 (Merity et al., 2017), using its official train/validation split. The test split is not used. We tokenize the non-empty lines with the Pythia tokenizer, join them with an end-of-text token, and cut the stream into 512-token blocks. This gives 119.9M training and 252K validation tokens. Every evaluation uses the same first 196,608 validation tokens. We report validation perplexity: the exponential of the average per-token cross-entropy loss, where lower is better.

We use AdamW (Loshchilov and Hutter, 2019) ($\beta = 0.9, 0.95$; weight decay 0.1; gradient clipping 1.0) with base learning rate 6×10^{-4} , for 4,000 steps of 16,384 tokens (65.5M tokens, less than one epoch, so no training block is seen twice). The learning rate $g(t)$ rises linearly over the first 200 steps and then follows a cosine decay. As a sanity check, the model can memorize 8 fixed training batches (loss 10.93 $\rightarrow$ 0.011), so the training code

works.

5.2 From measurements to a schedule

Each Phase 2 schedule changes the learning rate of a layer at one *switch step*. We take this step from Phase 1: it is the time at which that layer’s attention matrices got halfway to their final direction (t_{50}) in Pythia-70m, the same model size we train. Pythia trained for 143,000 steps and our models for 4,000, so we keep the same fraction of training. For example, layer 0’s attention reached halfway after about 11,200 of 143,000 steps (7.9%), so layer 0 switches at step 314 of 4,000. This gives a switch near step 310 for layers 0–2 and between steps 650 and 860 for layers 3–5, so deeper layers switch later. In each layer, the attention and the FFN switch at the same step.

5.3 Arms

We compare several versions of training, which we call *arms*. All arms use the setup above and differ only in their learning rate: each arm multiplies $g(t)$ by a factor $m(t, \ell)$ for the attention or FFN parameters of layer ℓ . Embeddings and layer norms always use $g(t)$.

- **baseline**: $m = 1$ everywhere.
- **attn-first**: attention uses $m = A$ before its layer’s switch point and $m = B$ after, with $B = A/3$; the FFN gets the mirror image.
- **attn-second**: the mirror of attn-first (FFN emphasised first).
- **attn-first-lrm, attn-second-lrm**: the same profiles, with A and B chosen so that each group’s total learning rate $\sum_t g(t) m(t, \ell)$ equals the baseline’s.
- **alternating**: the emphasis swaps between attention and FFN every 200 steps. It uses no Phase 1 input and tests whether separating the two groups helps by itself.

- **attn-freeze**: attention’s learning rate is set to 0 after its switch point. This tests freezing directly and is not budget-matched.

In attn-first and attn-second, A and B were chosen so that the *average* multiplier over training steps is 1, as in the baseline. This does not give each group the same total learning rate, because $g(t)$ is not flat. The group emphasised early, when g is largest, receives 9.1% more total learning rate than in the baseline, and the other group 4.6% less. We added the two -1rm arms after finding this imbalance. They remove it exactly, which separates the effect of timing from the effect of the budget.

Why emphasis instead of freeze-then-unfreeze.

Originally we planned to freeze the parts of a block that settle later, train only the parts that settle first, and unfreeze the rest once those had stabilized. Phase 1 removed the basis for this: at 70m attention and the FFN settle at the same time, and no matrix stops changing at a clear point that could trigger the unfreeze. We therefore changed how strongly each group is trained over time instead of switching groups off, and kept one freezing arm (attn-freeze) to measure the possible compute saving.

5.4 Comparing the arms

Each arm is trained several times with different random *seeds*. The seed fixes the random starting weights and the order in which training batches are drawn, so different seeds give different but equally valid runs. We use six seeds (0–5) for the baseline and the four directed arms (attn-first, attn-second and their -1rm versions), and three seeds (0–2) for alternating and attn-freeze, 36 runs in total. Six is the smallest number of seeds for which a result where every seed agrees can reach $p < 0.05$ (see below); the two side arms got three to save compute. For a given seed, every arm starts from the same initialization and sees the same batches in the same order; only the learning-rate factors differ.

We compare each arm to the baseline paired by seed. To test whether an arm differs from the baseline, we take the difference (arm minus baseline) for each seed. If the arm had no effect, each difference would be equally likely to be positive or negative, so we flip the signs of the differences in all possible ways and count how often the average is at least as far from zero as the real one; that fraction is the p -value. When all seeds agree in direction, this gives the smallest possible p : $2/64 = 0.031$ with six seeds, and $2/8 = 0.25$ with three, which

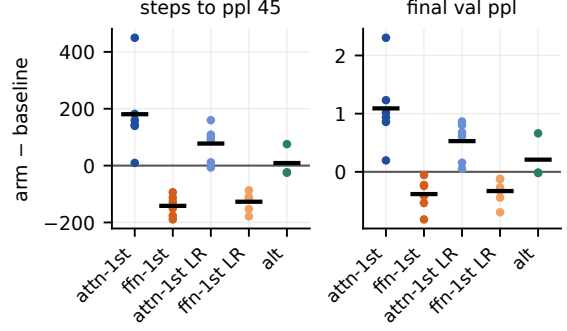


Figure 2: Per-seed differences from the baseline (each point is one seed; the bar is the mean). Left shows difference in step count to reach perplexity 45. Right shows difference in final perplexity value. Below zero means the arm beat the baseline for that seed. “LR” marks the learning-rate-matched arms. attn-freeze (+13 perplexity) is left out for scale.

can never be significant. A p of 0.031 therefore means “every seed agreed”. The main metrics are the number of steps to reach validation perplexity 45 and the final validation perplexity. The target 45 is set by rule, as the lowest round number reached by every run of baseline, attn-first and attn-second. p -values are not corrected for the several arms and metrics, so we rely mainly on whether all seeds agree in sign.

5.5 Results

Table 3 lists every arm, and Figure 2 shows the per-seed differences from the baseline.

Training the FFN first helps. Emphasising the FFN before the switch step makes training faster in every seed: attn-second needs 4.9% fewer steps to reach perplexity 45 and ends with slightly lower perplexity. This arm also gives the FFN more total learning rate than the baseline (Section 5.3), so the gain could have come from the extra learning rate alone. To check this, attn-second-1rm uses the same timing with the total learning rate matched to the baseline. It keeps almost all of the gain (4.4% fewer steps), again in every seed. What helps is therefore *when* the FFN is emphasised, not how much learning rate it gets.

Training attention first hurts. The mirror image, attn-first, is slower than the baseline in every seed. About half of this penalty came from the learning-rate imbalance: with the total matched, attn-first-1rm is still worse, but by roughly half as much, and no longer in every seed. Compared di-

arm	seeds	final val ppl	steps to 45	sec to 45	train s	peak GiB
baseline	6	40.54 ± 0.33	$2,915 \pm 48$	381	523	7.40
attn-first	6	41.63 ± 0.81	$3,096 \pm 165$	405	523	7.40
attn-second	6	40.16 ± 0.15	$2,774 \pm 23$	363	524	7.40
attn-first-lrm	6	41.07 ± 0.60	$2,993 \pm 103$	389	520	7.40
attn-second-lrm	6	40.21 ± 0.17	$2,788 \pm 22$	365	523	7.40
alternating	3	40.65 ± 0.56	$2,914 \pm 85$	385	530	7.40
attn-freeze	3	53.91 ± 0.56	—	—	512	7.40

Table 3: Phase 2 results, mean $\pm$ SD over seeds. “steps to 45” and “sec to 45” are the training step and training seconds at which validation perplexity first reaches 45 (interpolated; — if never reached). “train s” is total training time and “peak GiB” the peak GPU memory. At a given seed every arm shares the baseline’s initialization and data order.

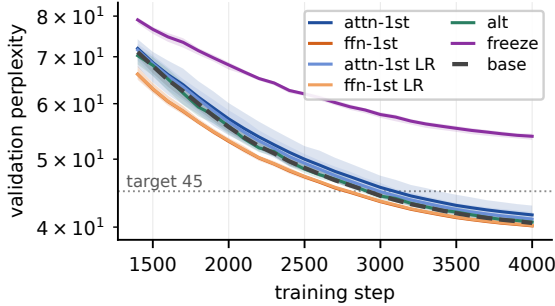


Figure 3: Validation perplexity over the last two thirds of training (line: mean over seeds; band: min–max). The baseline is dashed; alternating lies on it. Both FFN-first arms (orange) stay below it.

rectly, the two matched arms still differ in every seed, so the order in which the two groups are emphasised matters.

How large are these effects? Small. The best arm saves about 4.4% of the training steps and improves the final perplexity by under 1%. Every seed agrees on the direction, so the effects are consistent, but they are about how fast the model trains, not how good it becomes.

Alternating has no effect. alternating is indistinguishable from the baseline ($+0.21$ perplexity, $p = 1$; $+9$ steps, $p = 1$; 3 seeds) and its curve lies on the baseline’s (Figure 3). Changing the emphasis without the Phase 1 timing does not help.

Freezing hurts and saves little. attn-freeze sets attention’s learning rate to zero after each layer’s switch point, which falls 8–21% of the way into training. Attention’s total weight displacement drops to 0.536, against 1.025 in the baseline, so attention really stops learning. Final perplexity is 53.91 (33.3% worse), and the target of 45 is never reached. Training time drops by 2.5% (13.13 s of 523 s, 3 seeds).

6 Discussion

What Phase 1 shows. We found two clear orderings. Across layers, attention settles bottom-up at every model size. Inside a block, the attention output matrix W_O settles before the other attention matrices, clearly so at 410m. Between attention and the FFN the picture depends on scale: at 70m and 160m they settle together, and at 410m the FFN tends to settle first, though only under the halfway measure. This is not the “routing before memory” order we expected, and the query/key pair, which our H3 predicted would settle first based on the QK/OV split of Elhage et al. (2021), does not.

The order of emphasis matters for training speed. In Phase 2, emphasising the FFN before attention trained faster than the baseline in every seed, and emphasising attention first was slower. The learning-rate-matched arms show that this comes from the timing of the emphasis, not from giving the FFN more learning rate, and the alternating control shows that switching emphasis by itself does not help. A simple per-layer change of learning rates, with no extra compute, consistently reached the target about 4% sooner. It is also the same order that Phase 1 hints at for the largest model.

What remains open. The gain is small and is about training speed rather than final quality. We also cannot yet say why it works. The switch points came from 70m, where attention and the FFN settle together, so the gain may simply mean that the FFN benefits more from a high learning rate early in training. Arms with deliberately wrong switch points, or a Phase 2 at 410m, would tell these explanations apart.

Why freezing saved so little. Reflecting upon the underwhelming results, we believe that even

though freezing removes a matrix’s weight gradient and optimizer update, it does not remove the backward pass through that matrix, which earlier layers still need, or the activations stored for it. The possible saving was also small from the start: attention is 9% of the parameters, its optimizer state is allocated before the freeze, and peak memory is dominated by the output logits over a 50K vocabulary. So the result shows that freezing single matrices buys little in this setting, not that freezing never helps.

7 Limitations

- **The final checkpoint is not a finished model.** Pythia’s training ends at step 143,000, so we use that checkpoint as the “final” direction. The matrices are still changing slightly there (about 1% per 1000 steps), so the true end state may differ.
- **Limited model sizes.** We study three Pythia models, trained with one recipe, and train our own models only at the 70m size. The 36 training runs took about six GPU-hours on a single consumer GPU; repeating them at 410m would take many times longer, which did not fit the time and hardware available to us. Whether the schedules help at larger sizes, and whether our findings hold for other model families, remains untested.
- **Small numbers of seeds and layers.** Six seeds per main arm (three for the side arms) and only six layers in the smallest model limit how small an effect we can detect, and several Phase 1 results hold under only one of our two measures.
- **Learning rate and weight decay change together.** In PyTorch’s AdamW implementation, which we use, weight decay is multiplied by the learning rate, so a per-group multiplier also changes that group’s weight decay. Part of an effect could come from regularization.
- **Simplifying assumptions.** Several steps rely on simple rules rather than measurements. We move switch steps from Pythia’s 143,000-step run to our 4,000-step runs by keeping the same fraction of training, which assumes both runs progress at the same relative pace. Settling times between checkpoints are estimated by drawing a straight line between the two nearest checkpoints. Each matrix is treated as a single direction, and each layer is summarized

by the median of its matrices. The emphasis ratio of 3 and the use of attention’s switch step for both groups were fixed choices that we did not vary. We also combine all attention heads of a layer into one W_Q , W_K and W_V , so differences between individual heads are not visible.

8 AI Disclosure and Reflection

The research question, the two-phase design and the interpretation of the results are ours, and we take responsibility for everything in this paper. We used Claude (Anthropic; Opus 5 and Opus 5.5, through the Claude Code tool) to write code quickly, to learn background we did not know well such as Transformer internals and training procedures, to rapidly develop code, tests and logging, and to help draft and edit the paper text. We also used it to find the paper references for specific claims or techniques we used (and we’ve made sure to verify each quoted paper). We also wanted to adhere to professional standards in our statistical methodology, having well defined hypotheses and clear p values so we can make substantiated claims - the AI guided us on which correct statistical tools to use and how. These made the implementation much faster, but its output still had to be checked. We verified all the code written and didn’t blindly rely on the AI at any point.

9 Conclusion

We measured when each weight matrix of a Transformer block settles during pretraining, and used these measurements to build training schedules. We found that attention matrices settle bottom-up at all three model sizes, and that inside the attention block the output matrix W_O settles first. Attention and the FFN settle together in the smaller models, while in the largest the FFN tends to settle first. Weight change slows gradually rather than stopping at a clear point.

In training, the order of emphasis mattered. Emphasising the FFN early reached the target perplexity 4.4% faster than the baseline in every seed, with no extra compute and even with each group’s total learning rate matched, while emphasising attention early was slower. The effect is small, and why it works is still open: the schedule came from the 70m model, where attention and the FFN settle together. Freezing attention, in contrast, saved only 2.5% of training time and no memory, at a large cost in perplexity.

References

- Alessandro Achille, Matteo Rovere, and Stefano Soatto. 2019. Critical learning periods in deep networks. In *International Conference on Learning Representations (ICLR)*.
- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, et al. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.
- Sidney Black, Stella Biderman, Eric Hallahan, et al. 2022. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*.
- Andrew Brock, Theodore Lim, J. M. Ritchie, and Nick Weston. 2017. FreezeOut: Accelerate training by progressively freezing layers. *arXiv preprint arXiv:1706.04983*.
- Yixiong Chen, Alan Yuille, and Zongwei Zhou. 2023. Which layer is learning faster? A systematic exploration of layer-wise convergence rate for deep neural networks. In *International Conference on Learning Representations (ICLR)*.
- Nelson Elhage, Neel Nanda, Catherine Olsson, et al. 2021. A mathematical framework for transformer circuits. *Transformer Circuits Thread*.
- Jonathan Frankle, David J. Schwab, and Ari S. Morcos. 2020. The early phase of neural network training. In *International Conference on Learning Representations (ICLR)*.
- Mor Geva, Avi Caciularu, Kevin Ro Wang, and Yoav Goldberg. 2022. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Daniela Gottesman, Alon Gilaie-Dotan, Ido Cohen, et al. 2026. LMEnt: A suite for analyzing knowledge in language models from pretraining data to representations. *Transactions of the Association for Computational Linguistics*.
- Jeremy Howard and Sebastian Ruder. 2018. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. 2019. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
- Yuhan Liu, Saurabh Agarwal, and Shivaram Venkataraman. 2021. AutoFreeze: Automatically freezing model blocks to accelerate fine-tuning. *arXiv preprint arXiv:2102.01386*.
- Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*.
- Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. 2023. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*.
- Catherine Olsson, Nelson Elhage, Neel Nanda, et al. 2022. In-context learning and induction heads. *Transformer Circuits Thread*.
- Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. 2017. SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Jianlin Su, Murtadha Ahmed, Yu Lu, et al. 2024. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568.
- Kushal Tirumala, Aram H. Markosyan, Luke Zettlemoyer, and Armen Aghajanyan. 2022. Memorization without overfitting: Analyzing the training dynamics of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Minjia Zhang and Yuxiong He. 2020. Accelerating training of transformer-based language models with progressive layer dropping. In *Advances in Neural Information Processing Systems (NeurIPS)*.